FINAL REVIEW / INTERVIEW STUDY SERIES

Oral Drill — Technical Interview Practice

16 MAIN SECTIONS • PRINT-READY EDITION

In this guide

1. Why must equals and hashCode agree or why must you override both equals and hashCode? 2
2. Why is volatile count++ unsafe? 7
3. When does ConcurrentHashMap still need higher-level coordination? 10
4. What is the difference between thenApply and thenCompose? 13
5. Why can JDBC block a Vert.x event loop? 15
6. What does Spring Boot add to Spring? 18
7. Why might @Transactional not take effect? 20
8. How do you detect and fix N+1 queries? 28
9. How do you prevent lost updates? 29
10. Why can a LEFT JOIN unexpectedly remove unmatched rows? 31
11. How do you invalidate Redis data safely enough for the use case? 32
12. What must you validate in a JWT besides its signature? 34
13. How do you prevent cross-tenant access? 36
14. What does a payment timeout actually tell you? 42
15. What did you personally implement in your strongest project? 44
Practice checklist 45

Use this guide: Speak the **30–60 second answer** first. Use the **Follow-up** and **Small example** when asked. Open **Additional details** to understand the topic deeply. Each question also contains a **Full original notes** section: your source material is preserved (including original examples, diagrams and drafts).

Content integrity: All 15 original question headings and the complete body of your original notes appear below. The short spoken answers and further examples come from the earlier expanded study guide. For questions 8, 9, 12, 14 and 15, the original had no answer; added technical material is labeled and Q15 remains a personal template. The original source is preserved even where it contains code formatting or typing errors; organized code examples above it are corrected.

1. Why must equals and hashCode agree or why must you override both equals and hashCode?

30–60 second answer

`equals()` defines whether two Java objects are logically the same, while `hashCode()` helps hash-based collections locate them. The contract says that **if two objects are equal, their hash codes must also be equal**. The reverse is not true: different objects can share a hash code, which is called a collision. `HashMap` and `HashSet` first use the hash to narrow the search, then use `equals()` to confirm a match. If I override `equals()` but keep an incompatible `hashCode()`, two logically equal objects may end up in different buckets. That can cause failed lookups or duplicate entries in a set. I therefore implement both using the same stable equality fields.

Follow-up

What if two unequal objects have the same hash code?

That is allowed. A collision means the collection must compare candidate entries with `equals()` to tell them apart. It can reduce performance, but does not by itself break correctness. A **changing hash code** is a different problem: if an object is already a key in a `HashMap`, changing a field used by `equals()` and `hashCode()` may make it unreachable via normal lookup. Prefer immutable keys, and make sure equality is reflexive, symmetric, transitive and consistent. An object's hash is not required to stay the same between separate JVM executions.

Small example

JAVA

```java
record Person(String name) {} // Generates consistent equals and hashCode

Set<Person> people = new HashSet<>();
people.add(new Person("Alice"));
people.add(new Person("Alice"));
System.out.println(people.size()); // 1
```

Both records represent the same name, so the set keeps one entry. **Remember:** equal ⇒ same hash; same hash ⇏ equal.

Additional details

The contract

- `equals()` checks logical equality; `hashCode()` helps hash collections find the right bucket.
- **Equal objects must have equal hash codes.** Different objects can share a hash code; this is a collision, not a violation.
- `HashMap` first locates candidate entries with the hash, then compares with `equals()`.

What breaks if you override only `equals()`?

The original `Person` example compares objects by `name`, but inherits `Object.hashCode()`. Two separate objects named Alice can satisfy `p1.equals(p2)` while producing different hash codes. A `HashSet` may then store both; a `HashMap` lookup using an equivalent key may fail.

Correct implementation from your notes

JAVA

```java
import java.util.Objects;

class Person {
    String name;

    Person(String name) { this.name = name; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Person other)) return false;
        return Objects.equals(name, other.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }
}

Person p1 = new Person("Alice");
Person p2 = new Person("Alice");
System.out.println(p1.equals(p2)); // true
System.out.println(p1.hashCode() == p2.hashCode()); // true

Set<Person> people = new HashSet<>();
people.add(p1);
people.add(p2);
System.out.println(people.size()); // 1
```

Lookup flow

EXAMPLE

```
hashCode() -> candidate bucket -> equals() -> actual matching entry
```

The original examples, including the incorrect version and the longer flow diagram, are preserved below. The snippets above fix line-wrap errors in the original source.

Full original notes (preserved)

Source wording for question 1

The contract between `equals()` and `hashCode()`

If two objects are equal according to `equals()`, they must have the same `hashCode()`.

The reverse is not required: Two objects can have the same hash code but still be unequal. This is called a **hash collision**.

EXAMPLE

```
a.equals(b) == true
    => a.hashCode() == b.hashCode()
```

Hash-based collections like HashMap and HashSet rely on both methods to find and identify objects correctly.

While `equals()` defines logical equality, `hashCode()` helps hash-based collections locate objects efficiently.

Their contract ensures that logically equal objects can be found in the same bucket and correctly identified by `equals()`.

What happens if you violate the contract?

Suppose you create a Person class that overrides equals () but not hashCode ().

JAVA

```java
class Person {
    String name;

    Person(String name) {
        this.name = name;
    }

    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Person other)) return false;
        return name.equals(other.name);
    }
}
```

Now consider:

JAVA

```
Person p1 = new Person("Alice");
Person p2 = new Person("Alice");

System.out.

println(p1.equals(p2)); // true
        System.out.

println(p1.hashCode() ==p2.

hashCode()); // Usually false
```

Why? Because the inherited `Object.hashCode()` generally uses object identity rather than the name field. The two objects are logically equal but may have different hash codes.

The consequence in a `HashSet`:

JAVA

```
Set<Person> people = new HashSet<>();

people.

add(p1);
people.

add(p2);

System.out.

println(people.size()); // Can be 2, not 1
```

You expected one person because both objects represent Alice. But the set may put them in different hash buckets and fail to recognize that they are equal.

Similarly, a `HashMap` may fail to retrieve a value using a different but logically equal key.

The correct implementation:

Override both methods using the same fields that define logical equality.

JAVA

```
import java.util.Objects;

class Person {
    String name;

    Person(String name) {
        this.name = name;
    }
```

```
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Person other)) return false;

        return Objects.equals(name, other.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }
}
```

Now:

JAVA

```
Person p1 = new Person("Alice");
Person p2 = new Person("Alice");

System.out.

println(p1.equals(p2)); // true
        System.out.

println(p1.hashCode() ==p2.

hashCode()); // true

Set<Person> people = new HashSet<>();
people.

add(p1);
people.

add(p2);

System.out.

println(people.size()); // 1
```

How HashMap uses both methods?

EXAMPLE

```
1. Calculate hashCode()
Determine the hash bucket to search.
    |
    v
2. Find candidate entries
Use the hash to narrow down the search.
    |
    v
```

```
3. Compare with equals()
Determine whether a candidate key is actually equal.
```

If equal objects produce different hash codes, the map may search the wrong bucket and never compare those objects using equals ().

Official references: Java `Object` contract.

---

2. Why is volatile count++ unsafe?

30–60 second answer

`volatile` gives Java threads visibility and memory-ordering guarantees, but it does **not** turn a compound operation into one atomic action. `count++` is really three steps: read the current value, add one, and write the result. Two threads can both read five, both calculate six, and both write six. One increment is lost, even though the field is volatile. To implement a shared counter safely, I would use `AtomicInteger.incrementAndGet()` or protect the increment with `synchronized`. `volatile` is useful for a simple state flag that one thread writes and other threads read, but it is not enough for a read-modify-write operation.

Follow-up

When would you choose `AtomicInteger` versus `synchronized`?

Choose `AtomicInteger` for a single atomic numeric variable, including increments and compare-and-set. Use `synchronized` or another lock when **several fields must change together** and the whole invariant needs protection. For a highly contended statistics counter, `LongAdder` can offer better throughput, but `sum()` is not an atomic snapshot under concurrent updates. A shutdown flag is different: `volatile boolean shutdown` is suitable when workers only need to see its latest written state. Do not use `final` for a field you intend to increment or change.

Small example

JAVA

```java
AtomicInteger count = new AtomicInteger();

// Two threads each call this 1,000 times:
count.incrementAndGet();
// After both threads complete, count.get() is 2,000.
```

Remember: visibility is not atomicity.

Additional details

Why the increment is unsafe

`count++` is a **read → add one → write** sequence. If two threads read `5` before either writes, they may both write `6` instead of reaching `7`. `volatile` does not make those three steps indivisible.

What `volatile` guarantees

- **Visibility:** Another thread can observe a write to a volatile field.
- **Ordering:** Volatile reads and writes have Java memory-ordering effects.
- **Not atomicity:** It does not protect a read-modify-write sequence.

Three choices from your notes

Choice	Use when	Code
`AtomicInteger`	One shared counter	`count.incrementAndGet()`
`synchronized`	Protect a complete critical section	`public synchronized void increment() { count++; }`
`volatile` flag	Visibility of a simple state change	`private volatile boolean shutdown = false;`

JAVA

```java
private final AtomicInteger count = new AtomicInteger(0);
public void increment() { count.incrementAndGet(); }

// Alternative in a class, not mixed with the AtomicInteger example:
private int otherCount = 0;
public synchronized void incrementOther() { otherCount++; }
```

Source-code correction: The original note says `final int count = 0; count++;` and `final boolean shutdown = false`. Both use `final` incorrectly for a value that needs to change. These corrected versions retain the intended teaching point.

Full original notes (preserved)

Source wording for question 2

Interview response: volatile guarantees visibility, not atomicity.

Since count++ is a `read-modify-write` operation, multiple threads can overwrite each other's updates.

Use AtomicInteger or synchronization when you need a correct shared counter.

Explanation:

Consider this Java code:

```java
private final int count = 0;

// Called by multiple threads
count++;
```

Although count++ looks like one operation, it actually involves three steps:

1. Read the current value of count.
2. Increment the value by one.
3. Write the new value back to count.

These steps are not atomic.

In Java, volatile provides two important guarantees:

- **Visibility:** When one thread writes a new value to a volatile variable, other threads can observe that write.
- **Ordering:** It establishes memory-ordering guarantees around volatile reads and writes.

However, it does not make compound operations such as count++ atomic.

Think of it this way:

- `volatile` makes individual reads and writes visible.
- `synchronized` or `atomic` classes can make the entire increment operation safe.

How to fix it?

Option A: Use AtomicInteger (usually the simplest choice)

```java
import java.util.concurrent.atomic.AtomicInteger;

private final AtomicInteger count = new AtomicInteger(0);

public void increment() {
    count.incrementAndGet();
}
```

`incrementAndGet()` performs the increment atomically, so concurrent updates are not lost.

Option B: Use synchronized

```java
```

```
private int count = 0;

public synchronized void increment() {
    count++;
}
```

Only one thread at a time can execute this synchronized method on the same object, preventing concurrent increments from interfering.

Option C: Use volatile alone

JAVA

```
private final boolean shutdown = false;
```

This is appropriate when you need visibility for simple reads and writes, such as a shutdown flag:

Official references: Java `AtomicInteger` API.

---

3. When does ConcurrentHashMap still need higher-level coordination?

30–60 second answer

`ConcurrentHashMap` makes its individual map operations thread-safe, and methods such as `putIfAbsent()` and `computeIfAbsent()` give atomic operations for a mapping. But that does not mean an entire business workflow is atomic. I still need higher-level coordination when correctness depends on **multiple map calls, multiple keys, external state, or a consistent view of the whole map**. For example, checking `containsKey()` and then calling `put()` leaves a race between those calls. I would replace that with `putIfAbsent()` or `computeIfAbsent()`. For a transfer between two account balances, I would use a suitable lock or a database transaction so both changes succeed or fail together.

Follow-up

Why doesn't `computeIfAbsent()` solve every race?

It makes the computation and insertion atomic **for the relevant map key**; it is not a transaction spanning several keys or a database. Its mapping function should be short, and it must not modify that same map while computing. The function can also fail, leaving the mapping absent. If initializing a resource also writes to a database and publishes an event, I need an explicit workflow: for example, a state machine, coordinated transaction, or reliable outbox. Bulk traversal of a concurrently changing map also does not guarantee one globally consistent snapshot.

Small example

JAVA

```java
// Race-prone:
if (!map.containsKey("job-1")) map.put("job-1", new Job());

// Atomic insertion of a single key:
map.computeIfAbsent("job-1", key -> new Job());
```

Remember: thread-safe operations do not imply a thread-safe sequence.

Additional details

Map safety versus workflow safety

`ConcurrentHashMap` protects its internal map operations. It cannot make an unrelated sequence or business invariant atomic.

All four situations from your notes

1. **Check then act:** `containsKey()` followed by `put()` has a gap where another thread can act. Use `putIfAbsent()` or `computeIfAbsent()` as appropriate.
2. **Multiple keys:** A bank transfer updates two account balances. Each map operation can be safe while the overall transfer is inconsistent; use coordination or a transaction.
3. **Global invariant:** Checking total allocation and adding to `map` and `totalAllocated` must be one coordinated operation to enforce a capacity limit of 100.
4. **Multi-step lifecycle:** Initializing a service, registering it and publishing status needs a lock, `Future`, or explicit state machine when it must happen once.

JAVA

```java
// Check-then-act race:
if (!map.containsKey(key)) {
    map.put(key, createValue());
}

// Atomic computation for one mapping:
map.computeIfAbsent(key, k -> createValue());
```

Keep `computeIfAbsent()` mapping functions short; do not modify other mappings in the same map from the function. Neither this method nor the map itself coordinates external state.

Full original notes (preserved)

Source wording for question 3

Interview Response:

ConcurrentHashMap protects its internal data structure and provides atomic operations at the mapping level.

Higher-level coordination is needed when correctness depends on :

1. multiple operations
2. multiple keys,
3. external state, or
4. consistent global view.

Common situations requiring higher-level coordination

1. **Check-then-act operations**

   Even though every map operation is thread-safe, the entire check-and-update sequence is not atomic.

   ```java
   if (!map.containsKey(key)) {
     map.put(key, createValue());
   }
   ```

   Another thread can insert the key between the check and the insertion.

   Use `putIfAbsent()` or `computeIfAbsent()` when appropriate:

   ```java
   map.computeIfAbsent(key, k -> createValue());
   ```

   > Note: The mapping function should be short and should not attempt to update other mappings in the same map.

2. **Multiple keys must change together**

   Suppose a bank transfer moves money between two accounts. Each update may be atomic for its own key, but the transfer as a whole isn't atomic. You may need a shared lock, a transaction mechanism, or another design that coordinates both accounts.

3. **A global invariant must remain true**

   Imagine a system where the sum of all allocations must never exceed 100: Making map a `ConcurrentHashMap` does not coordinate the map update with `totalAllocated` or make the capacity check atomic.

   ```java
   if (totalAllocated < 100) {
       map.put(id, allocation);
   ```

```
        totalAllocated += allocation;
    }
```

4. **A multi-step lifecycle must happen only once**

   For example, multiple threads might attempt to initialize a service, register a resource, and publish its status. Atomicity of one map entry doesn't automatically coordinate all those steps.

   Use an appropriate lifecycle mechanism, such as a Future, a lock, or an explicit state machine.

Official references: Java `ConcurrentHashMap` API.

---

4. What is the difference between thenApply and thenCompose?

30–60 second answer

Both methods continue a `CompletableFuture` after an earlier stage produces a result. I use `thenApply()` when I want to **transform a value into another ordinary value**. For example, take a user's name and convert it to uppercase. I use `thenCompose()` when the next function **starts or returns another asynchronous operation**. It flattens the nested future so the result is one `CompletableFuture` rather than a future containing another future. A simple rule is: `thenApply` is like `map`; `thenCompose` is like `flatMap`. Neither method automatically makes blocking code non-blocking; I still need to consider which thread executes the callback.

Follow-up

When would you use `thenCombine()` instead?

Use `thenCompose()` when the second operation **depends on the result of the first**, such as fetching an account after fetching the account ID. Use `thenCombine()` when two futures can be initiated independently and you want to combine their results once both complete, such as retrieving a price and an exchange rate in parallel. Also distinguish `thenApply()` from `thenApplyAsync()`: the latter schedules the function with an asynchronous execution facility, or a supplied executor. An `Async` suffix is not a license to run blocking work on an inappropriate executor.

Small example

JAVA

```java
CompletableFuture<String> name = CompletableFuture.completedFuture("Sam");

CompletableFuture<String> upper = name.thenApply(String::toUpperCase);

CompletableFuture<User> user = name.thenCompose(this::fetchUserAsync);
// fetchUserAsync(String) returns CompletableFuture<User>
```

Remember: `thenApply` transforms; `thenCompose` chains and flattens.

Additional details

Two continuation methods

- `thenApply()` accepts the preceding value and returns a new ordinary value. Its result is a `CompletableFuture<T>`.
- `thenCompose()` accepts the preceding value and returns another `CompletionStage<T>`; composition flattens the nested future.

Original `thenApply()` example, formatted

JAVA

```java
CompletableFuture<String> future =
    CompletableFuture.supplyAsync(() -> "Hello")
        .thenApply(s -> s + " World");

System.out.println(future.join()); // Hello World
```

Follow-on async operation

JAVA

```java
CompletableFuture<User> user =
    fetchUserIdAsync().thenCompose(id -> fetchUserAsync(id));
```

Choose the method based on **what the continuation returns**, not just whether it runs after another stage. The earlier draft included the first example and the flattening rule; the second code example is an added illustration.

Full original notes (preserved)

Source wording for question 4

In Java's CompletableFuture, the main difference is:

- `thenApply()` transforms the result of a future into another value.

JAVA

```java
CompletableFuture<String> future =
        CompletableFuture.supplyAsync(() -> "Hello")
                .thenApply(s -> s + " World");

System.out.

println(future.join()); // Hello World
```

- `thenCompose()` transforms the result into another future and flattens it into a single future.

Official references: Java `CompletableFuture` API.

5. Why can JDBC block a Vert.x event loop?

30–60 second answer

Vert.x event loops are designed to handle many events with a small number of threads. They work well only when handlers return quickly. Traditional JDBC is a blocking API: executing a query or waiting for a connection can hold the calling thread until the operation finishes. If I call JDBC directly in an event-loop handler, that loop cannot process other requests or timers while it waits. Even a normal database delay can therefore hurt many unrelated requests. I would run blocking JDBC on a worker thread with `executeBlocking()`, or preferably use an asynchronous Vert.x SQL client where appropriate. I would also avoid calling `join()` or `get()` from the event-loop thread.

Follow-up

Does wrapping JDBC in `CompletableFuture.supplyAsync()` fix the problem?

Only if the JDBC work actually runs on a suitable executor **and the event loop never blocks waiting for it**. Calling `supplyAsync(...).join()` from the event loop still blocks that loop. Using the common pool for a large volume of blocking database work can also exhaust the pool's useful capacity, so executor and connection-pool sizing matter. Vert.x `executeBlocking()` places blocking work on a worker pool and returns a future; callbacks can continue asynchronously. A reactive SQL client avoids occupying a thread while the request waits for a database response, but the database query can still be slow.

Small example

JAVA

```java
vertx.executeBlocking(() -> jdbcQuery())
    .onSuccess(result -> request.response().end(result))
    .onFailure(err -> request.response().setStatusCode(500).end());
```

Remember: moving work off-loop helps only if you do not wait on-loop.

Additional details

Why it hurts an event loop

A Vert.x event loop serves many connections using relatively few threads. It needs each handler to return quickly. JDBC waits synchronously for a connection or query to finish; delays can come from the pool, locks, network latency, and the database query itself.

Option A: Offload blocking JDBC

JAVA

```java
vertx.executeBlocking(() -> jdbcQuery())
    .onSuccess(result -> {
        // Continue on the original Vert.x context.
        request.response().end(result);
    })
    .onFailure(err -> request.response().setStatusCode(500).end());
```

Vert.x runs the blocking call on a worker thread and completes the returned future on the original context.

Option B: Use a reactive SQL client

JAVA

```java
pool.preparedQuery("SELECT name FROM users WHERE id = ?")
    .execute(Tuple.of(userId))
    .onSuccess(rows -> {
        // Continue without waiting synchronously on the event loop.
    });
```

Trap: asynchronous work followed by a blocking wait

JAVA

```java
CompletableFuture.supplyAsync(() -> jdbcQuery()).join();
```

Although `jdbcQuery()` may execute on another thread, `join()` blocks the thread calling it. If that is an event loop, the problem remains. Compose callbacks instead of waiting.

Consequences and diagnosis

- **Latency:** unrelated requests wait while the loop is blocked.
- **Throughput:** fewer requests per second are handled.
- **Scalability:** widespread blocked event loops can make the service unresponsive.
- **Diagnostic clue:** Vert.x can log blocked-thread warnings, such as `vertx-eventloop-thread-3 has been blocked`.

Full original notes (preserved)

Source wording for question 5

How the Vert.x event loop works

A Vert.x event loop typically handles events for many connections on a small number of threads. As such, they depend on handlers returning quickly so it can continue processing other requests, timers, and I/O events.

Traditional JDBC operations are synchronous (blocking). So, even if the database is healthy, network latency, connection-pool waits, locks, or a slow query can keep the thread blocked.

The problem isn't necessarily the database being slow. It's that the event loop is waiting synchronously.

How to fix it?

Option A: Use executeBlocking ()

EXAMPLE

```
vertx.executeBlocking(() -> {
    return jdbcQuery(); // Runs on a worker thread
}).onSuccess(result -> {
    // Handle result on the original Vert.x context
    request.response().end(result);
}).onFailure(err -> {
    request.response().setStatusCode(500).end();
});
```

This keeps the JDBC call off the event loop. Vert.x runs the blocking callable in its worker pool and completes the returned future on the original context.

Option B: Use a reactive database client

For a more naturally non-blocking architecture, use an appropriate Vert.x reactive database client, such as a reactive SQL client supported for your database.

EXAMPLE

```
pool.preparedQuery("SELECT name FROM users WHERE id = ?")
    .execute(Tuple.of(userId))
    .onSuccess(rows -> {
        // Handle results without blocking the event loop
    });
```

Here, the database operation is asynchronous: the event loop can process other events while waiting for the database response.

Does wrapping JDBC in a CompletableFuture solve it?

Not by itself.

This is still dangerous when called from a Vert.x event loop:

EXAMPLE

```
CompletableFuture.supplyAsync(() -> jdbcQuery())
   .join();
```

Although `supplyAsync()` usually runs the JDBC operation on a different thread, `join()` blocks the calling thread while waiting for the result. If that calling thread is an event loop, you still have the same problem.

Instead, compose asynchronous stages without blocking: This avoids blocking the event loop, provided the callback itself doesn't perform blocking work.

EXAMPLE

```
vertx.executeBlocking(() -> jdbcQuery())
.onSuccess(result -> {
// Continue asynchronously
});
```

Why does this matter for performance?

Latency: Other requests on the blocked event loop wait longer.

Throughput: Fewer requests can be processed per second.

Scalability: If all event loops become blocked, the entire Vert.x instance can become unresponsive.

Diagnostics: Vert.x can log warnings such as vertx-eventloop-thread-3 has been blocked

Official references: Vert.x Core Manual — Don't block the event loop.

6. What does Spring Boot add to Spring?

30–60 second answer

Spring is the underlying framework for dependency injection, application components, web development and other infrastructure. Spring Boot builds on it to reduce setup and provide a simpler way to run applications. Its main additions are **auto-configuration**, **starter dependencies**, **embedded server support**, **externalized configuration**, and **production features through Actuator**. For example, adding a web starter lets Boot configure much of a typical web application based on the dependencies present, while allowing me to override defaults. I can often package and run the application directly as an executable JAR. Boot does not replace Spring; it makes common Spring application setups faster and more consistent.

Follow-up

How does auto-configuration know what to create?

Boot evaluates conditions, including which classes and beans exist and which configuration properties are set. If relevant libraries are available and I have not supplied my own bean, Boot may configure a default. If I define a compatible explicit bean, the auto-configuration can back off. `@SpringBootApplication` enables key

application setup, including auto-configuration and component scanning. Starter dependencies help assemble compatible libraries; they are not themselves the runtime mechanism that creates every bean. Actuator exposes operational features such as health and metrics, subject to dependencies and endpoint exposure settings.

Small example

JAVA

```java
@SpringBootApplication
public class App {
    static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

With appropriate web dependencies, Boot can set up an embedded web server and the surrounding web infrastructure.

Additional details

What Spring Boot adds to Spring

Spring remains the core framework. Boot reduces setup and deployment work through defaults and conventions.

Addition	Meaning	Example or outcome
Auto-configuration	Configures components based on available libraries and application settings	Beans for an appropriate web stack
Starters	Groups useful dependencies	`spring-boot-starter-web`
Embedded servers	Packages the app with a server when applicable	Run a Spring MVC service using `java -jar` with embedded Tomcat
Actuator	Adds production-oriented health, metrics and diagnostic endpoints	Health checks
External configuration and conventions	Supports properties, YAML, environment and CLI arguments with overridable defaults	`application.yaml`

What to emphasize in an interview

Boot **does not replace Spring**. It makes Spring applications quicker to set up, run and operate. A starter supplies dependencies; auto-configuration is the separate mechanism that chooses configuration based on conditions.

Full original notes (preserved)

Source wording for question 6

Spring Boot makes Spring applications easier to configure, run, and deploy.

Spring provides the core framework and features; Spring Boot adds sensible defaults, automatic configuration, and tools that reduce the amount of manual setup.

There are five major additions.

1. **Auto-configuration** Spring Boot automatically configures beans and components based on the libraries on your classpath and the settings you provide.
2. **Starter dependencies** Starters bundle commonly used dependencies so you don't have to select every library individually.

   For example, `spring-boot-starter-web` brings together the dependencies commonly needed for a Spring MVC web application.
3. **Embedded servers** Boot can package an application with an embedded server, such as Tomcat, so you can run it directly with java -jar instead of deploying a WAR to a separately installed server.
4. **Production-ready features** Spring Boot Actuator provides features such as health checks, application metrics, and diagnostic endpoints, depending on configuration.
5. **Externalized configuration and conventions** Boot makes it convenient to configure applications through `application.properties`, `application.yaml`, environment variables, and command-line arguments. It also provides sensible defaults that you can override.

Official references: Boot auto-configuration · Boot Actuator.

7. Why might @Transactional not take effect?

30–60 second answer

The first thing I check is whether the method call went through a **Spring-managed transaction proxy**. In Spring's default proxy mode, a method calling another annotated method on the same object bypasses the proxy, so the inner annotation does not start a new transactional boundary. I also check that Spring created the bean, the method can be intercepted, and the correct transaction manager is configured. Sometimes the transaction exists but the result looks wrong because of rollback rules: by default, Spring rolls back for unchecked exceptions and errors, not ordinary checked exceptions. Finally, asynchronous work or operations on unrelated database connections may be outside the transaction.

Follow-up

How would you debug a transaction that unexpectedly committed?

I would check whether the method was invoked through a proxy and whether the transaction was active on that thread. Then I would inspect the exception: was it checked, caught and swallowed, or deliberately configured not to roll back? For a checked exception, I can use `@Transactional(rollbackFor = Exception.class)` when that behavior is required. I would also verify propagation settings such as `REQUIRED` versus `REQUIRES_NEW`, the chosen transaction manager, and whether the database work uses an enlisted connection. These are separate failure modes; an annotation alone cannot make unrelated external services part of one transaction.

Small example

JAVA

```java
@Service
class OrderService {
    void placeOrder() { save(); }     // Self-call bypasses proxy advice

    @Transactional
    public void save() { /* persist order */ }
}
```

Move `save()` to an injected Spring-managed `OrderWriter` service and call it through that bean, or design the public transactional boundary on `placeOrder()` itself.

Additional details

Troubleshoot in this order

Your original notes identify **10 separate reasons**. Group them by the kind of failure so you can debug quickly without forgetting any.

A. The transactional advice never runs

1. **Self-invocation:** A method calls another method in the same object (`saveOrder()` or `this.saveOrder()`); the call skips the Spring proxy.
2. **Method/class not proxyable:** Standard Spring proxies cannot advise private or static methods; class-based proxies cannot override final methods or subclass final classes. Visibility support depends on proxy strategy and Spring version.
3. **Bean created with `new` :** It is not the proxied, Spring-managed instance.
4. **Transaction management misconfigured:** Check transaction enablement and the actual bean and proxy configuration.

JAVA

```java
@Service
class OrderService {
    public void processOrder() { saveOrder(); } // Self-call; skips proxy advice

    @Transactional
```

```
    public void saveOrder() { /* database operations */ }
}
```

Fix: Call the transactional method through a Spring-managed service, or define the transaction on an eligible proxied public entry method.

JAVA

```
@Service
class OrderWriter {
    @Transactional
    public void saveOrder() { /* database operations */ }
}

@Service
class OrderService {
    private final OrderWriter writer;
    OrderService(OrderWriter writer) { this.writer = writer; }
    public void processOrder() { writer.saveOrder(); }
}
```

B. It runs but does not roll back as expected

1. **Exception/rollback rules:** Spring normally rolls back for `RuntimeException` and `Error`, but not checked exceptions by default. An exception that is caught and swallowed may lead to a commit.
2. **Wrong transaction manager:** With multiple data sources, the selected manager might not protect the repository in question.
3. **Propagation settings:** `REQUIRED`, `REQUIRES_NEW`, `SUPPORTS`, `NOT_SUPPORTED`, and `NEVER` affect whether work joins, starts, suspends, or avoids a transaction.
4. **Operation outside the transaction:** Different connections, unenlisted resources, an unsupported storage engine, autocommit, replicas or a later async action can make a correctly active transaction appear ineffective.

JAVA

```
// Roll back on a checked exception if that is the required policy:
@Transactional(rollbackFor = Exception.class)
public void saveOrder() throws IOException {
    repository.save(order);
    throw new IOException("Failed");
}

// Select the manager that protects the intended resource:
@Transactional("ordersTransactionManager")
public void saveOrderWithManager() { /* ... */ }

// Explicitly run without a transaction:
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void updateOrder() { /* ... */ }
```

Propagation at a glance

Setting	Behavior
`REQUIRED`	Join existing or start a new transaction
`REQUIRES_NEW`	Suspend existing and start a new transaction
`SUPPORTS`	Join if there is one; otherwise proceed without
`NOT_SUPPORTED`	Suspend existing; proceed without one
`NEVER`	Fail if a transaction exists

C. The boundary or framework does not match

1. **Async/thread boundary:** Imperative Spring transactions are generally thread-bound. `@Async` work does not automatically share the caller's transaction; create an appropriate transaction in the worker if needed. Reactive code uses a reactive transaction manager and context.
2. **Wrong annotation/infrastructure:** Distinguish Spring's `org.springframework.transaction.annotation.Transactional`, Jakarta's `jakarta.transaction.Transactional`, and legacy `javax.transaction.Transactional`. Check JDBC/JPA, JTA or reactive setup.

Other original cautions retained: Catching and logging an exception without rethrowing can commit; neither Spring annotations nor propagation automatically make separate databases or external services one atomic transaction. The full original code and troubleshooting list appear below.

Full original notes (preserved)

Source wording for question 7

Spring's transaction management is usually proxy-based. The method must be invoked through the `Spring-managed proxy` for the transaction advice to run.

Here are the most common causes, how to identify them, and how to fix them.

1. **Self-invocation (the most common cause)** Calling a transactional method from another method in the same class directly using `this` and bypasses the Spring proxy.

JAVA

```java
@Service
public class OrderService {

    public void processOrder() {
        saveOrder(); // Self-invocation: transaction may not start
    }

    @Transactional
    public void saveOrder() {
```

```
            // Database operations
        }
    }
```

Fix: Move transactional method into another Spring-managed service and inject it, or call the method through a proxy.

JAVA

```
@Service
public class OrderWriter {

    @Transactional
    public void saveOrder() {
        // Database operations
    }
}

@Service
public class OrderService {

    private final OrderWriter orderWriter;

    public OrderService(OrderWriter orderWriter) {
        this.orderWriter = orderWriter;
    }

    public void processOrder() {
        orderWriter.saveOrder(); // Goes through the proxy
    }
}
```

2. **The method is not eligible for proxy interception**

   Depending on the proxy mechanism and Spring version, method visibility and class structure can prevent interception.

   - **Private methods:** Cannot be intercepted by standard proxy-based transaction management.
   - **Final methods:** Cannot be overridden by class-based proxies, so transaction advice cannot be applied to them.
   - **Final classes:** Cannot be subclassed by class-based proxies, although interface-based proxies may still work.
   - **Static methods:** Are not intercepted by standard Spring transaction proxies.

   Fix: Use an eligible, typically public, instance method on a non-final Spring-managed class. Package-private and protected methods can work with class-based proxies in modern Spring versions, but behavior depends on the proxy strategy and configuration.

3. **The object was created with new** Spring can apply transactions only when the invocation passes through the relevant Spring proxy.

JAVA

```
OrderService service = new OrderService(); // Not Spring-managed
service.saveOrder();
```

Fix: Let Spring create and inject the bean. Inject it into the calling component instead of constructing it manually.

JAVA

```
@Service
public class OrderService {

    @Transactional
    public void saveOrder() {
        // ...
    }
}
```

4. **The exception does not trigger a rollback**

   By default, Spring rolls back on `RuntimeException` and `Error`, but not on **checked exceptions**.

JAVA

```
@Transactional
public void saveOrder() throws IOException {
    repository.save(order);
    throw new IOException("Failed");
}
```

The transaction may commit despite the checked exception, depending on how the exception is handled and transaction completion proceeds.

Fix: Specify rollback rules when needed.

JAVA

```
@Transactional(rollbackFor = Exception.class)
public void saveOrder() throws IOException {
    repository.save(order);
    throw new IOException("Failed");
}
```

Also watch for exceptions that are caught and swallowed:

JAVA

```
@Transactional
public void saveOrder() {
    try {
        repository.save(order);
```

```
            performOperation();
        } catch (Exception e) {
            log.error("Operation failed", e);
            // No exception escapes; Spring may commit
        }
    }
```

If the transaction should roll back, rethrow the exception or explicitly mark the transaction rollback-only.

5. **Transaction management is not configured correctly**
6. **The wrong transaction manager is being used**

Applications with multiple data sources or transaction managers can accidentally start a transaction against the wrong resource.

JAVA

```java
@Transactional("ordersTransactionManager")
public void saveOrder() {
    // ...
}
```

Fix: Specify the correct transaction manager explicitly, if necessary, and verify that the repository and transaction manager operate on the intended data source.

Also, transactions do not automatically span arbitrary independent databases or external services. That requires appropriate distributed transaction infrastructure or a different consistency strategy.

1. **Transaction propagation changes the behavior**

Propagation settings can cause a method to join an existing transaction or start a separate one.

- `REQUIRED` — joins an existing transaction or creates one.
- `REQUIRES_NEW` — suspends an existing transaction and starts a new one.
- `SUPPORTS` — participates in a transaction if one exists, but does not require one.
- `NOT_SUPPORTED` — executes without a transaction, suspending an existing one if applicable.
- `NEVER` — requires that no transaction exists.

For example:

JAVA

```java
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void updateOrder() {
    // Executes without a transaction
}
```

Fix: Check the propagation setting on both the caller and the called method, not just the annotation on the method you're debugging.

1. **The transaction is active, but the data is not rolled back**

Sometimes `@Transactional` works correctly, but the observed behavior makes it appear ineffective.

Common reasons include:

- The operation uses a different database connection or data source.
- A database operation is performed through a resource not enlisted in the transaction.
- The database or storage engine does not support the expected transactional behavior.
- The code commits manually or uses autocommit outside Spring's transaction management.
- The code reads from a replica or another connection and observes different state.
- The transaction commits before an asynchronous operation executes.

Fix: Verify the actual connection, transaction manager, database resource, and transaction boundaries. Do not assume that annotating a method makes every operation it invokes transactional.

1. **Async execution or thread boundaries**

Imperative Spring transactions are generally thread-bound. They do not automatically propagate to another thread.

JAVA

```
@Transactional
public void processOrder() {
    asyncService.processAsync(); // Runs on another thread
}
```

If `processAsync()` uses @Async, its work generally runs outside the caller's transaction unless it establishes its own transaction.

Fix: Put `@Transactional` on the method executed by the async worker if that work needs a transaction. For reactive applications, use the appropriate reactive transaction manager and preserve the reactive execution context.

1. **The wrong annotation or framework is being used**

Make sure the annotation matches the transaction infrastructure:

- `org.springframework.transaction.annotation.Transactional` — Spring's annotation.
- `jakarta.transaction.Transactional` — Jakarta Transactions annotation, supported in appropriate Spring configurations.
- `javax.transaction.Transactional` — legacy Java EE annotation, relevant to older stacks.

Also verify whether you are using Spring JDBC/JPA, JTA, or reactive persistence, since their transaction requirements differ.

Official references: Spring transaction annotations and proxy behavior.

8. How do you detect and fix N+1 queries?

30–60 second answer

N+1 is a database access pattern where I load N parent records with one query, and then issue an additional query for each parent's related data. For example, loading 100 orders and lazily reading each order's customer can result in 101 queries. To detect it, I enable SQL logging or query-count metrics and observe how the query count grows when the number of returned records grows. To fix it, I choose a fetching strategy that matches the use case: a join fetch or entity graph for the needed relations, batch fetching for multiple lazy associations, or a projection that selects only required columns. I verify the improvement with a realistic dataset.

Follow-up

Why not make every relationship eager?

Eager loading can over-fetch unnecessary data and may still result in extra SQL selects, depending on the query and ORM fetching strategy. Fetch-joining several to-many associations can multiply rows and complicate pagination. I prefer explicit fetching for a particular read path instead of making the whole model eager. For a list API, a DTO projection is often efficient because it avoids building a large object graph. I would compare SQL count, rows returned, total latency and memory, not just the number of queries. The correct solution depends on whether the relationship is to-one, to-many, paginated, or used by multiple endpoints.

Small example

JAVA

```java
// JPQL: load orders and their customer in one SQL join
@Query("select o from Order o join fetch o.customer")
List<Order> findOrdersWithCustomer();
```

For a **to-one** relation this can avoid per-order lazy loads. Test carefully before fetch-joining **to-many** collections with pagination.

Additional details

Added explanation — the original answer was blank.

Detect an N+1 pattern (added material; the original answer was blank)

1. Load the parent collection and record SQL/query-count metrics.

2. Access the related data for each parent; observe whether each access triggers another select.
3. Repeat with a larger result set: if query count grows approximately `1 + N`, you may have N+1.

Fix only the read path that needs it

- `JOIN FETCH` or an entity graph for required relations.
- Batch fetching for lazy relationships.
- DTO projection for list endpoints that need only a few fields.

Avoid changing all associations to eager fetching. To-many joins can multiply rows and interfere with pagination; compare query counts, total rows, memory and response time.

JAVA

```java
@Query("select o from Order o join fetch o.customer")
List<Order> findOrdersWithCustomer();
```

This is an illustration for a to-one relationship, not a universal solution.

Full original notes (preserved)

Source wording for question 8

Official references: Hibernate User Guide — fetching strategies.

---

9. How do you prevent lost updates?

30–60 second answer

A lost update occurs when two requests read the same value and later one write overwrites the other request's change. The common case is two users editing the same record based on an old version. My default choice for ordinary JPA entity updates is **optimistic locking** with a `@Version` column. The database update checks that the version has not changed; if another transaction already updated the row, the later update fails with a conflict instead of silently overwriting data. I can then ask the caller to reload or safely retry. For counters, I prefer an atomic database update, and for high-contention workflows I consider row locks.

Follow-up

Optimistic or pessimistic locking — which would you choose?

Optimistic locking is a good starting point when conflicts are infrequent, because reads do not require a long-held exclusive lock. A conflict becomes visible at update or commit and the application must decide whether a retry is safe. Pessimistic locking, such as `SELECT ... FOR UPDATE`, can be appropriate when a short transaction

must serialize access to a row, but it increases waiting and deadlock risk. For simple increments, a single SQL statement is often better than read-then-write. I also check transaction boundaries and isolation: simply putting several requests in transactions does not automatically prevent every lost-update scenario.

Small example

JAVA

```java
@Entity
class Product {
    @Id Long id;
    @Version Long version;
    int stock;
}
```

If two transactions loaded version 3, the first can update it to version 4; the second then gets an optimistic-lock conflict instead of silently replacing the first update.

Additional details

Added explanation — the original answer was blank.

What a lost update looks like (added material; the original answer was blank)

EXAMPLE

```
Read by A: stock = 10
Read by B: stock = 10
A writes 9
B writes 9 -> A's change is lost (expected 8)
```

Three approaches

Strategy	When useful	Trade-off
Optimistic locking (`@Version`)	Conflicts are uncommon	Detect conflict and retry or return an error
Pessimistic locking (`FOR UPDATE`)	Short, high-contention changes needing serialization	More waiting and possible deadlocks
Atomic SQL update	Counter or single-field adjustment	Operation must fit one statement

JAVA

```java
@Entity
class Product {
    @Id Long id;
    @Version Long version;
```

```
    int stock;
}
```

If two transactions loaded the same version, one update can advance the version and the other receives a conflict instead of silently overwriting. Do not blindly retry workflows that create side effects.

Full original notes (preserved)

Source wording for question 9

Official references: Hibernate optimistic locking.

---

10. Why can a LEFT JOIN unexpectedly remove unmatched rows?

30–60 second answer

A `LEFT JOIN` normally returns every row from the left table, even when there is no matching row on the right. For an unmatched row, columns from the right table become `NULL`. The surprise comes when I add a condition on the right table in the `WHERE` **clause**, such as `WHERE payment.status = 'PAID'`. For unmatched rows, that comparison is not true, so the `WHERE` clause removes them. The query then behaves like an inner join for that condition. If I want all left-side rows but only qualifying matches from the right table, I put the right-side filter in the `ON` **clause** instead.

Follow-up

When is a right-table condition in `WHERE` correct?

It is correct when the business question intentionally wants only rows that have a qualifying match. For example, "customers who have a paid order" legitimately excludes customers without a paid order. But "all customers, showing paid orders if any" requires a `LEFT JOIN` with the paid filter in `ON`. There is another useful pattern: `LEFT JOIN ... WHERE right.id IS NULL` to find left-side rows with **no matching right-side record**. I always translate the SQL back into the expected business result and test it with one matched row and one unmatched row.

Small example

SQL

```sql
-- Keeps customers even if they have no PAID order:
SELECT c.id, o.id
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.id AND o.status = 'PAID';
```

Remember: `ON` affects matching; `WHERE` filters the result after the join.

Additional details

Why unmatched rows disappear

`LEFT JOIN` preserves all left-table rows initially. When no right-side record matches, that side's columns are `NULL`. A right-table condition in `WHERE`, such as `o.status = 'PAID'`, removes those null-extended rows.

Compare the two queries

SQL

```sql
-- Keeps only customers with a paid order:
SELECT c.id, o.id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.status = 'PAID';

-- Keeps all customers; attaches paid orders when available:
SELECT c.id, o.id
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.id AND o.status = 'PAID';
```

The original note identifies `WHERE` versus `ON` as the key cause. Use `WHERE` when excluding nonmatches is intentional. Use `ON` for a condition that should restrict matched right-side rows without dropping left-side rows.

Full original notes (preserved)

Source wording for question 10

A LEFT JOIN can unexpectedly remove unmatched rows when a condition in the WHERE clause filters out the NULL values generated for rows that have no match on the right-hand table.

The most common cause is putting a condition on the right-hand table in WHERE instead of ON.

Official references: PostgreSQL table expressions and joins.

11. How do you invalidate Redis data safely enough for the use case?

30–60 second answer

I start with the business requirement: how stale can the data be, and what happens if it is wrong? For ordinary read-heavy data, I often use **cache-aside**: read from Redis first, load from the database on a miss, and store the value with a TTL. When a write succeeds in the database, I invalidate the specific Redis key so the next

reader reloads it. The TTL limits how long a forgotten or missed invalidation can persist. I also plan for races where a concurrent reader loads an old value and repopulates the cache after invalidation. For sensitive data, such as authorization or balances, I use stronger consistency controls or bypass the cache.

Follow-up

What is the invalidation race and how do you prevent it?

Imagine reader A sees a cache miss and starts loading version 1 from the database. Writer B commits version 2 and deletes the cache key. Then A finishes late and writes version 1 back into Redis. The key is stale **even though deletion happened after the database write**. Possible controls include a versioned cache key, storing and checking a record version, serializing loads and writes for the key, or using change events with carefully designed ordering. Exact choice depends on tolerated staleness. For hot keys, a short-lived request-coalescing mechanism can reduce stampedes. A TTL is a safety net, not proof of immediate consistency.

Small example

EXAMPLE

```
GET product:42 -> cache miss
DB SELECT product 42 -> price 20
SET product:42 (value=20, TTL=60s)

On successful price update:
DB UPDATE product 42 -> price 25
DEL product:42
```

This is the basic cache-aside flow; production code must address concurrent repopulation.

Additional details

First establish the acceptable consistency level

Your original note asks three questions:

1. **How stale may cached data be?**
2. **How costly is rebuilding it after invalidation?**
3. **What is the consequence of showing a briefly stale value?**

Your three rules, made concrete

- **Invalidate narrowly:** delete the exact affected key rather than a broad collection of unrelated keys.
- **Make staleness detectable:** include a version, freshness timestamp or controlled TTL when it helps the use case.
- **Prevent stale repopulation:** a concurrent read may load an old database value and write it into Redis after the writer invalidated the key.

Cache-aside example (added follow-up)

EXAMPLE

```
GET product:42 -> cache miss
DB SELECT -> value 20
SET product:42 to 20 with a TTL

DB UPDATE -> new value 25
DEL product:42 after successful commit
```

For stronger consistency, use a version check, ordered invalidation mechanism or a design that avoids caching sensitive values. A TTL limits staleness duration but does not eliminate write/read races.

Full original notes (preserved)

Source wording for question 11

The safest way to invalidate Redis data depends on

- **how stale the data can be**,
- **how expensive it is to rebuild**, and
- **what happens if a stale value is served briefly**.

The general principle is:

- invalidate narrowly,
- make stale data detectable, and
- ensure concurrent requests cannot repopulate the cache with outdated values.

Official references: Redis cache-aside pattern.

12. What must you validate in a JWT besides its signature?

30–60 second answer

A valid JWT signature only proves that the token was signed with an accepted key and has not been modified. It does **not** automatically mean the token should be accepted by my API. I also validate its expected **issuer**, **audience**, and time-based claims such as **expiration** and **not-before**. I restrict the allowed signing algorithms, select keys from a trusted source, and confirm the token type and intended use, such as access token versus ID token. Then I validate application-specific identity, scopes or roles and resource permissions. If my system supports revocation, I also check whether the token or session has been revoked.

Follow-up

Why is an audience check important?

An identity provider might issue tokens for several APIs. A token that has a valid signature could still be intended for a different service. Checking `aud` prevents another application's token from being accepted by mine. Similarly, `iss` binds the token to a trusted issuer, and the signing key must belong to that issuer. I must not let the token header arbitrarily choose an algorithm or an untrusted remote key source. `exp` and `nbf` are time validity checks; small clock-skew allowances may be configured. The claims identify and constrain the caller, but authorization must still be enforced at the requested resource.

Small example

JSON

```
{
  "iss": "https://auth.example.test",
  "aud": "orders-api",
  "sub": "user-123",
  "exp": 1900000000
}
```

The `orders-api` must reject this token if its issuer is untrusted, it is expired, or it is not an intended audience—even if its signature verifies.

Additional details

Added explanation — the original answer was blank.

Validate token context, not just cryptography (added material; the original answer was blank)

- **Key and algorithm:** use a trusted signing key and an explicitly allowed algorithm; reject untrusted key selection.
- **Issuer (`iss`):** matches the expected identity provider.
- **Audience (`aud`):** this API is an intended receiver.
- **Time claims:** check `exp` and `nbf` with the configured clock-skew policy.
- **Token type/use:** do not accidentally accept an ID token where an access token is required.
- **Identity and permissions:** validate required subject/claims and still authorize the exact requested action and resource.
- **Revocation/session state:** apply checks where supported by your system.

JSON

```
{"iss":"https://auth.example.test", "aud":"orders-api", "sub":"user-123", "exp":1900000000}
```

A correct signature with `aud = another-api` is **not** sufficient to access `orders-api`.

Full original notes (preserved)

Source wording for question 12

Official references: RFC 7519: JWT claims · RFC 8725: JWT security guidance.

13. How do you prevent cross-tenant access?

30–60 second answer

In a multi-tenant system, I apply **defense in depth**. I first authenticate the user, then verify that they belong to the tenant they want to access and have permission for the requested resource. The backend creates a trusted tenant context; it must not assume that a tenant ID sent in a header or URL is authorized. Every data-access operation must enforce that scope. In a shared PostgreSQL database, I would use tenant-filtered queries backed by row-level security where practical. I would also namespace cache keys and file paths by tenant, carry tenant context into background jobs, and run automated negative tests proving tenant A cannot access tenant B's data.

Follow-up

Is `WHERE tenant_id = ?` enough?

It is necessary in many designs, but not enough on its own. A developer could forget the predicate, a background job could reuse the wrong tenant context, or a cache key could omit the tenant. PostgreSQL row-level security adds a database enforcement layer. When using a transaction-scoped setting such as `app.tenant_id`, set it from an **already verified backend identity** inside the transaction and use a database role that cannot bypass RLS. Superusers and `BYPASSRLS` roles bypass those policies; table owners normally bypass them unless `FORCE ROW LEVEL SECURITY` applies. I would test reads, writes, files, caches and pooled connections.

Small example

SQL

```sql
-- tenant_id must come from trusted, authorized server context:
SELECT id, title
FROM documents
WHERE id = :document_id
  AND tenant_id = :verified_tenant_id;
```

Even if a user guesses another document ID, the query does not return data outside the verified tenant.

Additional details

The defense-in-depth rule from your notes

Authenticate the caller, authorize tenant membership and permissions, and enforce tenant isolation at every data boundary. **Never trust an unverified `tenant_id` sent by the frontend.**

Security controls across the full stack

Layer	What must be enforced
Authentication	Verify identity and session/token
Authorization	Verify tenant membership and resource permissions
API	Enforce verified tenant scope to prevent IDOR/BOLA
Database	Tenant-filtered queries and, when suitable, RLS or separated tenant databases
Cache	Tenant-prefixed keys
Object storage	Tenant-scoped access to files
Background jobs	Carry and validate the correct tenant context
Monitoring	Detect and log cross-tenant access attempts

Request flow and ownership of tenant context

EXAMPLE

```
Client request (may propose X-Tenant-ID)
    |
    v
Validate JWT / authenticate user
    |
    v
Verify selected tenant membership + resource permissions
    |
    v
Create trusted, server-side TenantContext
    |
    +--> API authorization
    +--> Tenant-scoped SQL / RLS
    +--> Cache and object storage
    +--> Background-job tenant context
    |
    v
Return only authorized data
```

Tenant selection can arrive through a token, session, URL or header; **authorization is decided by the backend**. A valid JWT does not authorize access to arbitrary tenant IDs.

Database isolation: unsafe versus safer

SQL

```sql
-- Unsafe: only a resource ID is checked.
SELECT * FROM documents WHERE id = :document_id;

-- Safer: verified tenant ID must also match.
SELECT * FROM documents
WHERE id = :document_id
  AND tenant_id = :verified_tenant_id;
```

PostgreSQL RLS as the second guard

SQL

```sql
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE documents FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON documents
USING (tenant_id = current_setting('app.tenant_id')::uuid)
WITH CHECK (tenant_id = current_setting('app.tenant_id')::uuid);
```

Set the tenant context from a **verified identity** within each transaction:

SQL

```sql
BEGIN;
SELECT set_config('app.tenant_id', :verified_tenant_id, true);
SELECT * FROM documents;
COMMIT;
```

The `true` argument makes it transaction-local. Use a role that cannot bypass RLS. PostgreSQL superusers and `BYPASSRLS` roles bypass policies; be mindful of table-owner behavior. Reused connections and background workers must not leak old tenant context.

Negative tests from your notes

- Tenant A cannot **read, update or delete** Tenant B's rows.
- Changing object ID or tenant ID does not bypass server authorization.
- Cache entries and stored files do not leak across tenants.
- Pooled database connections do not retain the previous tenant's context.
- Background jobs cannot execute with the wrong tenant.

The original recommended three-part baseline is **deny-by-default application authorization + database tenant scoping/RLS + automated negative tests in CI/CD**. Both original ASCII request diagrams and the more detailed RLS SQL are retained below.

Full original notes (preserved)

Source wording for question 13

Use defense-in-depth approach.

The core rule is:

Every request must be authenticated, authorized, and enforce tenant isolation at every data-access boundary.

Never rely on a single tenant_id filter or frontend access control.

1. **Key security controls**

Layer	Control	Purpose
Authentication	Validate identity and session	Identify the caller
Authorization	Verify tenant membership and permissions	Prevent unauthorized tenant access
API	Enforce tenant scope server-side	Prevent IDOR/BOLA attacks
Database	Row-Level Security (RLS) or tenant-isolated databases	Block cross-tenant queries
Cache	Namespace keys by tenant	Prevent cached data leakage
Storage	Tenant-scoped object access	Protect files across tenants
Background Jobs	Carry and validate tenant context	Prevent asynchronous data leakage
Monitoring	Log and alert on tenant access violations	Detect attempted or accidental breaches

1. **Enforce tenant isolation on every request** A secure request flow looks like this:

EXAMPLE

```
Client Request
      |
      v
Authenticate User
      |
      v
Resolve Tenant Context
      |
      v
Verify Tenant Membership
```

```
      |
      v
Check Resource Permissions
      |
      v
Apply Database Tenant Isolation
      |
      v
Return Authorized Data
```

Important: Never trust a tenant_id supplied by the client without verifying that the authenticated user is allowed to access that tenant.

2. **Example: Safe database access**

   Unsafe — resource ID only

```sql
SELECT *
FROM documents
WHERE id = :document_id;
```

   Safer — tenant-scoped query

```sql
SELECT *
FROM documents
WHERE id = :document_id
  AND tenant_id = :verified_tenant_id;
```

   The verified_tenant_id must come from a trusted, authorized server-side context.

   For PostgreSQL, add `Row-Level Security (RLS)` as another protection layer:

```sql
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE documents FORCE ROW LEVEL SECURITY;

CREATE
POLICY tenant_isolation
ON documents
USING (
  tenant_id = current_setting('app.tenant_id')::uuid
)
WITH CHECK (
  tenant_id = current_setting('app.tenant_id')::uuid
);
```

Set the tenant context from the verified identity within the database transaction. Use an application database role that cannot bypass RLS, and reset transaction context before reusing connections.

PostgreSQL documents important RLS exceptions, including superusers and roles with BYPASSRLS.

3. **Test cross-tenant isolation** At minimum, test these scenarios:
   - Tenant A cannot read Tenant B's records.
   - Tenant A cannot update or delete Tenant B's records.
   - Changing object IDs or tenant IDs does not bypass authorization.
   - Cache and file access cannot expose another tenant's data.
   - Reused database connections do not retain a previous tenant's context.
   - Background jobs cannot run under an incorrect tenant context.
4. **Recommended approach** For a SaaS application with a shared database, use three independent controls:
   1. **Application:** Centralized, deny-by-default authorization.
   2. **Database:** Tenant-scoped queries backed by RLS.
   3. **Testing:** Automated cross-tenant negative tests in CI/CD.

   The main principle is **tenant isolation must remain effective even when one application-layer check is missed.**

How tenant scope is set

Tenant scope is set by the **backend after authenticating the user and verifying their access to the tenant.**

The frontend may request a tenant, but it must never decide which tenant the user is authorized to access.

1. **Request flow**

EXAMPLE

```
Client Request
    |
    | Authorization: Bearer <JWT>
    | X-Tenant-ID: tenant_123
    v
API Gateway / Backend
    |
    v
1. Validate JWT
    |
    v
2. Identify user
    |
    v
3. Verify user belongs to tenant_123
    |
    v
4. Create trusted TenantContext
    |
    | user_id   = user_456
    | tenant_id = tenant_123
    | role      = admin
```

```
    v
5. Enforce tenant scope
    |
    +---- API authorization
    |
    +---- Database queries / RLS
    |
    +---- Cache and storage
    |
    v
Return tenant-specific data
```

2. **Where does tenant_id come from?**

   Select tenant from token, session, URL, or header. Store verified tenant_id server-side

   Important: A valid JWT proves the token is authentic. It does not automatically prove access to every tenant requested by the caller.

3. **How is tenant scope applied to the database?** For `PostgreSQL with Row-Level Security (RLS)`, tenant scope is typically set **per transaction**.

   POSTGRESQL

```postgresql
BEGIN;
-- Set verified tenant context
SELECT set_config(
'app.tenant_id',
:verified_tenant_id,
true
);

-- RLS automatically filters rows
SELECT * FROM documents;

COMMIT;
```

The `true` argument makes the setting transaction-local, preventing tenant context from unintentionally carrying into a later transaction on a reused connection.

This assumes RLS policies are configured and the database role cannot bypass them.

Official references: PostgreSQL row-level security.

14. What does a payment timeout actually tell you?

30–60 second answer

A payment timeout tells me that **my system did not receive a definitive response within the allowed time**. It does not prove that the charge failed. The payment provider may have received and completed the request, but the response could have been delayed or lost. Therefore, blindly retrying with a new request can create a

duplicate charge. I treat the payment state as **unknown or pending reconciliation**, not automatically failed. I use an idempotency key for safely retrying the same logical operation, check the payment status using the provider's authoritative API or supported webhooks, and record the final state reliably before telling the customer the outcome.

Follow-up

What would a reliable payment flow look like?

Before sending a create or capture request, persist an internal payment attempt with a unique identifier and idempotency key. Use the same key for retries of that same operation, following the provider's documented retention and retry rules. On timeout, do not initiate a fresh independent charge. Query the existing payment, receive provider webhook events with verified authenticity, deduplicate event handling and reconcile if the response remains uncertain. Model states such as `PENDING`, `SUCCEEDED` and `FAILED`, and define when the order may be fulfilled. Different gateways have different semantics, so I would follow the selected provider's API rather than assume all timeouts behave identically.

Small example

EXAMPLE

```
Order 77 -> payment attempt pay-77, idempotency key pay-77
POST charge request -> timeout
Do NOT submit a new charge under key pay-77-retry-2
Check status of pay-77; retry safely under the original key if supported
Mark SUCCEEDED or FAILED only after authoritative confirmation
```

Remember: timeout ≠ payment failure.

Additional details

Added explanation — the original answer was blank.

What the timeout means (added material; the original answer was blank)

The client did not receive a definitive response within the limit. The provider **may still have completed the payment**. Do not treat timeout as failed, and do not start a fresh independent charge without checking.

Safer workflow

1. Persist a local payment attempt and a stable idempotency key before calling the gateway.
2. On a timeout, mark the result **UNKNOWN/PENDING** for reconciliation.
3. Query the provider's authoritative status or consume verified, deduplicated webhooks.
4. If the provider supports safe retries, retry the same logical operation with the original idempotency key under its documented rules.
5. Fulfill the order only after a known final state.

EXAMPLE

```
Order 77 -> attempt pay-77 -> timeout
Do not issue an unrelated second charge
Reconcile existing attempt -> SUCCEEDED or FAILED
```

Specific idempotency-key retention, status codes and webhook semantics depend on the payment provider.

Full original notes (preserved)

Source wording for question 14

Official references: Stripe idempotent requests — example of provider-specific retry semantics. Different payment providers may differ.

15. What did you personally implement in your strongest project?

30–60 second answer

"My strongest project was **[project name or business problem]**. Its goal was to **[specific outcome for users or the business]**. I personally owned **[module, service or feature]**, rather than the whole project. I designed and implemented **[two specific technical changes]**, using **[actual technologies]**. One important challenge was **[real constraint, such as latency, consistency or security]**. I addressed it by **[your decision and why]**, and validated the result using **[test, production metric or review method]**. The outcome was **[measured result, or a precise qualitative result if no metric exists]**. My main learning was **[practical trade-off or lesson]**."

Follow-up

How do I know that was your contribution and not your team's?

Be concrete about the boundary of your work. Explain what you implemented yourself, what your teammates owned, and where you collaborated. Be ready to draw the request path, identify a failure mode, explain the alternative designs you rejected, and show how you tested the solution. Interviewers often look for one decision you can defend with a trade-off: for example, simplicity versus throughput or consistency versus latency. **Never invent percentage improvements or scale numbers**; if you did not measure them, describe the observable result and how you would measure it next time.

Small example

EXAMPLE

```
Problem:       [Users experienced duplicate work on retries]
My ownership:  [Implement idempotency check in one service]
```

```
Decision:      [Store request ID and result in a durable table]
Trade-off:     [Extra database write for safer retries]
Validation:    [Concurrent duplicate-request integration test]
Outcome:       [Repeated request reused the original result]
```

Remember: use **I** for your actual work and **we** for team accomplishments.

Additional details

Template only — no project history was supplied.

Make this answer about your actual contribution

The original file contains only this interview question; it provides **no project facts**. Keep the 30–60 second template above until you fill it with verified facts.

Prepare these exact details:

1. **Problem:** What user or business issue was your project solving?
2. **Ownership:** Which endpoint, component, workflow or feature did you personally implement?
3. **Design:** What two technical choices did you make and why?
4. **Challenge:** What broke or was risky, and how did you handle it?
5. **Validation:** What tests, logs or metrics proved it worked?
6. **Outcome:** What changed? Give a measured result only if it was actually measured.
7. **Trade-off:** What alternative did you reject, and what would you improve now?

Use **“I”** for your work and **“we”** for team results. Expect follow-up questions about failure modes, edge cases, test strategy and architecture.

Full original notes (preserved)

Source wording for question 15

Practice checklist

1. Answer the question directly in the first sentence.
2. Explain the mechanism in plain words, not only keywords.
3. State the failure mode or trade-off.
4. Give one concrete example only when helpful.
5. Stop at 60 seconds; keep the rest for the follow-up.
6. For question 15, fill in all placeholders and practice with your real project details.

Corrections made from the original draft

- Question 2: `final int count = 0; count++` was invalid Java; `final boolean shutdown = false` cannot later change. The revised examples use appropriate mutable or atomic state.
- Question 1: the original Java snippet had broken line breaks; it has been replaced with a smaller, valid record example.
- Questions 4, 8, 9, 12 and 14: expanded missing or incomplete explanations.
- Question 7: self-invocation is **one** common reason an annotation appears ineffective; a transaction can also be active but still commit because of rollback rules.
- Question 15: kept explicitly as a fill-in template rather than attributing invented experience to you.